

Báo cáo Review Code – Payment Module (SOLID Review)

1) Phạm vi review

Domain: Payment trong backend (services, controllers, models, clients, DTO)

Đường dẫn chính:

*AIMS/aims-backend/src/main/java/com/ecommerce/aims/payment/***

Mục tiêu: Đánh giá mức độ tuân thủ SOLID, xác định rủi ro thiết kế khi mở rộng provider (PayPal/VietQR/...), và đề xuất hướng refactor có thể triển khai theo từng bước.

2) Tóm tắt điều hành (Executive Summary)

Các vấn đề chính:

- **SRP vi phạm rõ:** PaymentService và RefundService đang vừa orchestration payment, vừa cập nhật trạng thái Order (cross-domain side effects).
- **OCP/DIP vi phạm rõ:** logic phụ thuộc if/else theo PaymentProvider và phụ thuộc trực tiếp vào class cụ thể (PayPalService, VietQRService) → thêm provider mới sẽ buộc sửa service trung tâm, khó test.
- **LSP/ISP chưa vi phạm trực tiếp** nhưng **nguy cơ cao:** response chung PaymentResultResponse có payload khác nhau theo provider; năng lực provider (initiate/capture/refund) không đồng nhất.

Tác động nếu không xử lý:

- Mỗi lần thêm provider / thay đổi flow sẽ “chạm” PaymentService/RefundService → tăng rủi ro regression.
- Unit test khó cô lập (mock nhiều nhánh), integration test phức tạp.
- Contract API dễ gây lỗi phía client (field có/không tùy provider).

Khuyến nghị chiến lược:

- Introduce **PaymentGateway abstraction + registry** (OCP/DIP).
- Tách **Order side-effects** sang OrderService hoặc event handler (SRP).
- Segregate theo **capabilities**: initiate/capture/refund (ISP).
- Chuẩn hóa **response contract** theo provider payload (LSP).

3) Chi tiết review theo SOLID

3.1) SRP – Single Responsibility Principle

3.1.1 PaymentService đang gộp nhiều trách nhiệm

Hiện trạng: createPayment(...) vừa:

- validate request
- load order
- tạo transaction
- chọn provider + gọi provider
- lưu transaction
- trả response

Ở markCaptured(...), service lại:

- load transaction
- capture theo provider hoặc tự set status

- save transaction
- **cập nhật Order** (updateOrderPaid(...))

Kết luận SRP: PaymentService đang làm **orchestration payment + business state transition của Order** → dễ phát sinh coupling giữa payment và order.

3.1.2 RefundService cũng trộn refund + order

Minh họa (RefundService#refund):

- load transaction
- gọi refund theo provider (PayPal)
- set status
- save
- **cập nhật order** (updateOrderRefunded(...))

Khuyến nghị SRP:

- Tách “cập nhật order status” sang:
 - OrderService (được gọi bởi application layer), hoặc
 - event-driven: publish PaymentCaptured, PaymentRefunded → handler cập nhật order.

3.2) OCP – Open/Closed Principle

3.2.1 Branch if/else theo provider trong PaymentService

Hiện trạng: PaymentService phải sửa code khi thêm provider mới.

```
if (request.getProvider() == PaymentProvider.PAYPAL) { ... }  
else if (request.getProvider() == PaymentProvider.VIETQR) { ... }  
else { ... }
```

3.2.2 RefundService phụ thuộc logic PayPal

```
if (transaction.getProvider() == PaymentProvider.PAYPAL) {  
    payPalService.refund(...);  
}
```

3.2.3 VietQRService map status hardcode

```
return switch (status) {  
    case PAID -> PaymentStatus.CAPTURED;  
    case CANCELLED, EXPIRED, FAILED, UNDERPAID -> PaymentStatus.FAILED;  
    default -> PaymentStatus.INIT;  
};
```

Khuyến nghị OCP:

- Introduce PaymentGateway (hoặc PaymentProviderHandler) + registry:

- Map<PaymentProvider, PaymentGateway>
- hoặc inject List<PaymentGateway> rồi build map trong constructor.

3.3) LSP – Liskov Substitution Principle (nguyên cơ thiết kế)

Hiện trạng: PaymentResultResponse dùng chung nhưng payload khác nhau:

- PayPal trả approvalUrl, captureUrl
- VietQR trả qrContent

PayPal:

```
.approvalUrl(approvalUrl)
.captureUrl(captureUrl != null ? captureUrl : ...)
```

VietQR:

```
.qrContent(transaction.getQrContent())
```

Rủi ro: client giả định field tồn tại đồng nhất → NPE/hiển thị sai/logic client phình to với if/else.

Khuyến nghị LSP (theo hướng contract rõ ràng):

- Option A: Response “envelope” + payload theo provider:
 - PaymentResultResponse { commonFields..., Map<String, Object> providerPayload }
- Option B: Polymorphic DTO (Jackson @JsonTypeInfo) theo provider.
- Option C: Tách endpoint/flow theo provider (ít linh hoạt hơn).

3.4) ISP – Interface Segregation Principle (nhu cầu rõ)

Dấu hiệu hiện có:

- PaymentService cần biết initiate/capture/refund trong khi **không phải provider nào cũng có đủ**.

Khuyến nghị ISP:

Tách interface theo capability:

- PaymentInitiator
- PaymentCapturer
- PaymentRefunder

Provider nào hỗ trợ gì thì implement cái đó; service trung tâm không ép phải có đủ.

3.5) DIP – Dependency Inversion Principle

Hiện trạng:

- PaymentService phụ thuộc class cụ thể:
 - `private final PayPalService payPalService;`
 - `private final VietQRService vietQRService;`
- RefundService phụ thuộc PayPalService

Tác động:

- Unit test khó: phải mock nhiều concrete services và nhánh điều kiện.
- Khó mở rộng provider: thay đổi ripple effect.

Khuyến nghị DIP:

- Inject abstraction PaymentGateway/capability interfaces và dùng registry.

4) Các điểm phù hợp SOLID (điểm mạnh hiện tại)

4.1 Tách lớp gọi API ra khỏi business service

Hiện tại đã tách PayPalClient và VietQRClient riêng để call API (tốt cho SRP và test theo tầng).

Minh họa (PayPalClient#createOrder): đóng gói payload + handle WebClient exception và throw BusinessException.

Điểm này nên được **giữ nguyên và mở rộng theo cùng pattern** khi refactor gateway.

5) Kế hoạch refactor theo giai đoạn (ít rủi ro, dễ merge)

Phase 1 – “No behavior change” (1–2 PR)

1. Introduce capability interfaces + registry.
2. Wrap PayPalService/VietQRService thành gateway implementations (adapter) **nhưng giữ logic cũ.**
3. Thay if/else trong PaymentService bằng registry.require(...).
4. Unit test cho registry + 2 providers.

Kết quả: OCP/DIP cải thiện ngay, hành vi không đổi.

Phase 2 – Tách Order side-effects (1–2 PR)

1. Tạo PaymentEventPublisher (hoặc Spring ApplicationEventPublisher).
2. PaymentService.markCaptured và RefundService.refund publish events thay vì updateOrderPaid/refunded.
3. Implement handler cập nhật order (ở order domain).

Kết quả: SRP sạch hơn, giảm coupling.

Phase 3 – Chuẩn hóa response contract (1 PR)

- Chọn một trong các option LSP (envelope/polymorphic).

- Update controller/docs/client mapping.

Kết quả: Giảm rủi ro client-side bug.

5) Khuyến nghị test & quality gates

Unit tests (bắt buộc):

- PaymentGatewayRegistry:
 - provider tồn tại → resolve đúng capability
 - provider không hỗ trợ capability → throw đúng exception
- PaymentService:
 - createPayment với PayPal/VietQR không còn branch
 - transaction lifecycle (INIT → CAPTURED/FAILED) đúng

Integration tests (khuyến nghị):

- Test H2/Postgres testcontainer + mock external clients (PayPalClient, VietQRClient) bằng stub.
- Idempotency cho capture/refund (nếu có retry).

Static quality gates:

- Checkstyle/SpotBugs (nếu đã dùng)
 - Enforce “no provider if/else” trong service trung tâm (code review rule)
-

8) Kết luận

- **SRP/OCP/DIP**: đang là điểm nghẽn chính trong payment domain; vấn đề thể hiện rõ ở PaymentService và RefundService qua orchestration + provider branching + order side-effects.
- **LSP/ISP**: hiện chưa “nở” vì số provider còn ít, nhưng khi mở rộng sẽ tạo nợ kỹ thuật lớn (response contract và capability mismatch).
- Đề xuất refactor theo 3 phase ở trên giúp **giảm rủi ro**, **giữ behavior ổn định**, và **mở rộng provider** nhanh hơn trong tương lai.